Luiss
Libera Università Internazionale
degli Studi Sociali Guido Carli

Project Example: CVE-2022-21662

Data Privacy and Security

Data Science and Management (DASMA)

LUISS

The Project

The project:

- is done individually
- involves the study of **one** real-world vulnerability
- is submitted as **one** 15 pages PDF report:
  - description of the affected program (e.g., architecture, components, implementation details, etc.)
  - description of the vulnerability type (e.g., recap of what is a SQL injection)
  - description of the vulnerability (e.g., how it works, where it is located in the code, etc.)
  - exploitation of the vulnerability (e.g., step by step for the reproduction, discussion of the exploit details, etc.)
  - reproduction of the vulnerability in a docker environment (e.g., detailed instructions to reproduce the vulnerability, output of the exploit, etc.)
  - mitigation of the vulnerability (e.g., how to fix the vulnerability, code changes, etc.)
  - reproduction of the fix in a docker environment (e.g., detailed instructions to reproduce the fix, output of the exploit, etc.)
- is evaluated based on a 0-30 points scale:
  - 26 points for the report
  - 4 points for reproduction environment

CVE Assignment

- Each student should pick one (or two when non compliant) CVE
  - Google query: **CVE *program that you want to consider***
  - E.g., "Wordpress CVE", "Flask CVE", "n8n CVE", ...
  - You can consider even vulnerability types that have not been covered in the course
  - Understand whether there is enough material to analyze:
    - Source code (if the application is open source)
    - Vulnerability description
    - Exploit code
    - etc.

How to choose CVE?

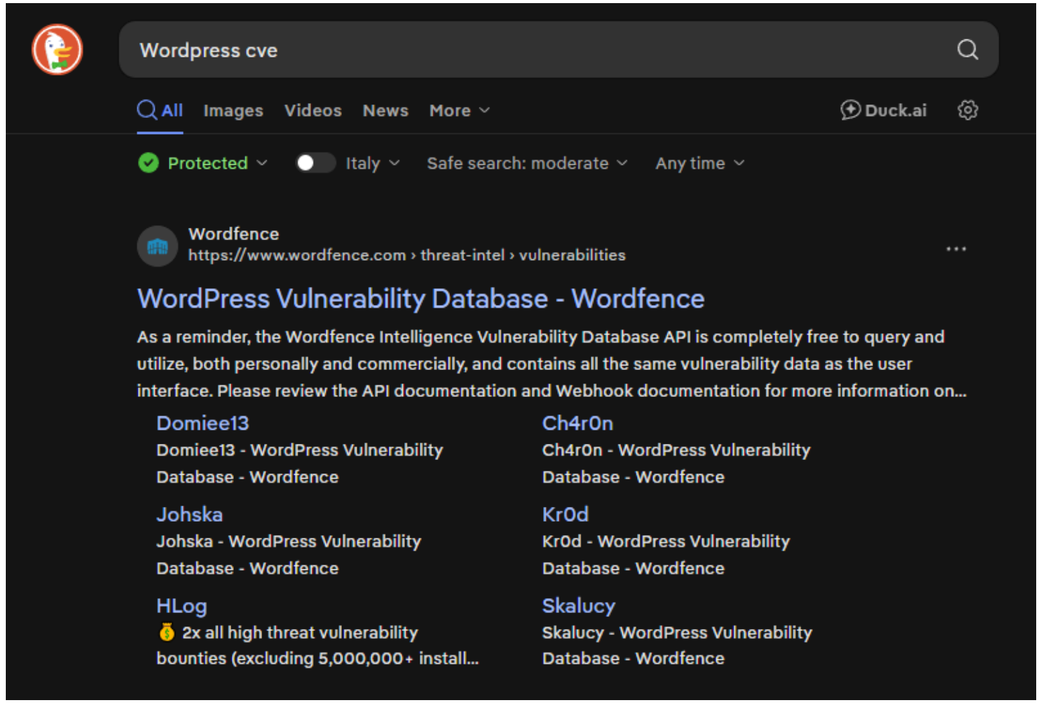
1. Think about a software that you like! For example, I like **Wordpress**.

How to choose CVE?

1. Think about a software that you like! For example, I like **Wordpress**.
2. Use your preferred search engine and find CVEs about **Wordpress**.

How to choose CVE?

1. Think about a software that you like! For example, I like **Wordpress**.
2. Use your preferred search engine and find CVEs about **Wordpress**.



How to choose CVE? - Part 2

3. Use a correct amount of time to choose a CVE:
    - If you choose a CVE and than you don't find any helpful material, the project WILL BE painful
    - If you choose a CVE that's too complex to replicate, the project WILL BE painful

How to choose CVE? - Part 2

3. Use a correct amount of time to choose a CVE:
    - If you choose a CVE and than you don't find any helpful material, the project WILL BE painful
    - If you choose a CVE that's too complex to replicate, the project WILL BE painful

> How to choose correctly?

Choosing a CVE

- Go back and forth multiple times between selecting a CVE and conducting basic research on it.

Choosing a CVE

- Go back and forth multiple times between selecting a CVE and conducting basic research on it.
- Use different search engines and different keywords to find CVEs

Choosing a CVE

- Go back and forth multiple times between selecting a CVE and conducting basic research on it.
- Use different search engines and different keywords to find CVEs
- Try to interrogate your preferred LLM and see what comes out

Choosing a CVE

- Go back and forth multiple times between selecting a CVE and conducting basic research on it.
- Use different search engines and different keywords to find CVEs
- Try to interrogate your preferred LLM and see what comes out

REMEMBER: you need to be able to discuss your CVE, understand how it works, how the exploit functions, how to fix it, and related aspects.

Select a CVE

Once you have reached a decision remember to:

- Check which CVEs have been already proposed by other students and thus cannot be considered by you: **URL**
- Submit your proposal via the form: **FORM**
- Check if your CVE has been approved: **URL**

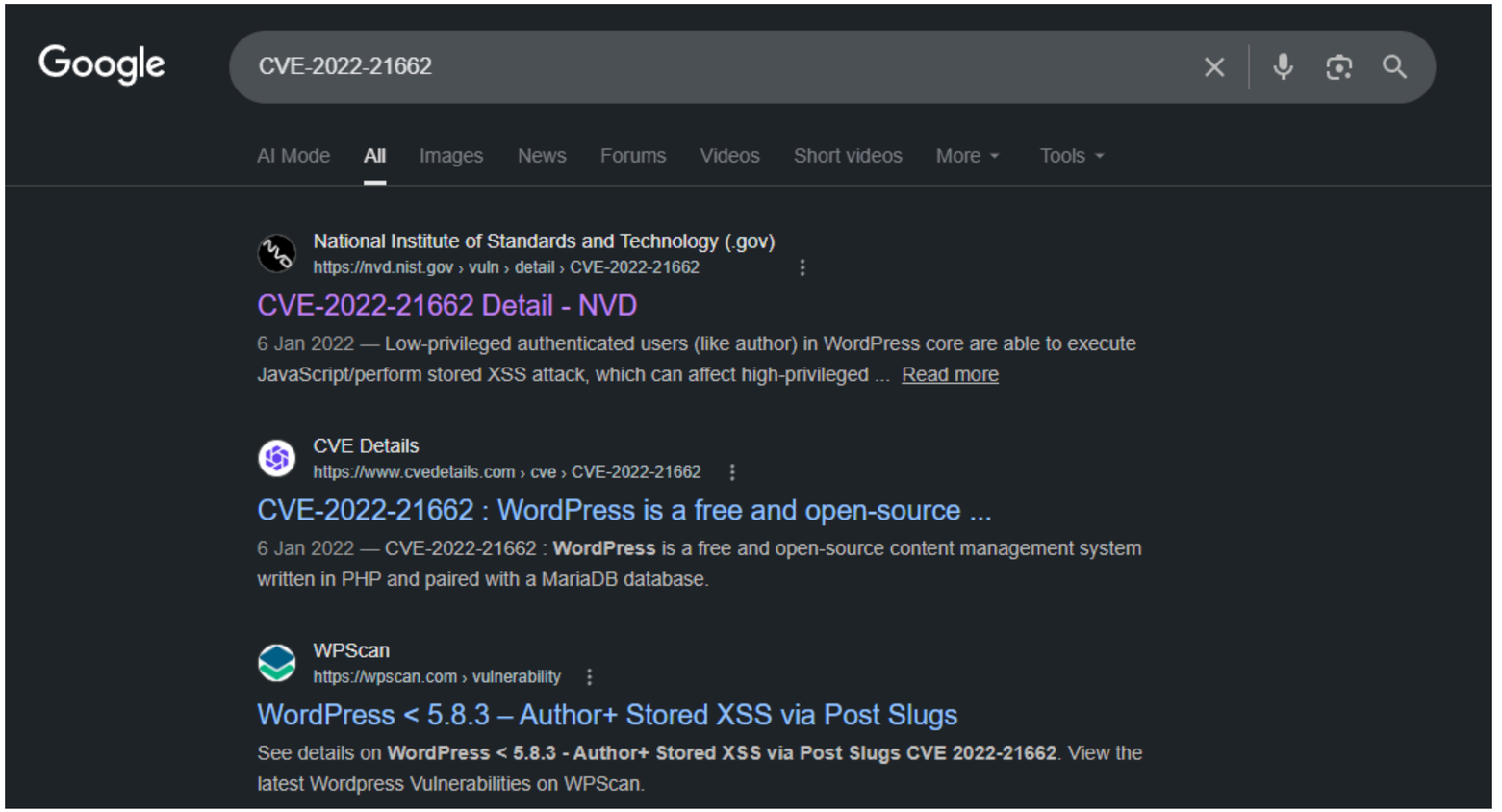
Select a CVE

Once you have reached a decision remember to:

- Check which CVEs have been already proposed by other students and thus cannot be considered by you: **URL**
- Submit your proposal via the form: **FORM**
- Check if your CVE has been approved: **URL**

> Once you proposal is approved, you can start working on the report(s).

CVE-2022-21662

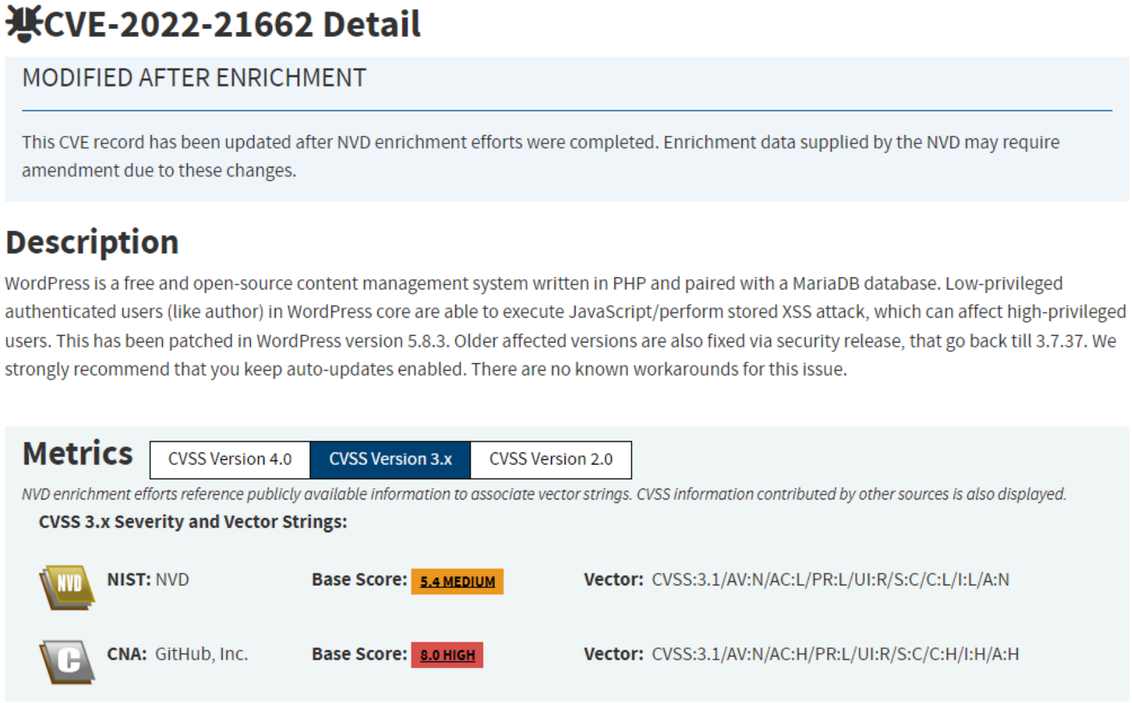


The NIST Website

- CVE numbers are assigned by CVE Numbering Authorities (CNAs).
- NIST's National Vulnerability Database (NVD) traditionally enriched and scored these vulnerabilities
- Trust the process!

The NIST Website

- CVE numbers are assigned by CVE Numbering Authorities (CNAs).
- NIST's National Vulnerability Database (NVD) traditionally enriched and scored these vulnerabilities
- Trust the process!



CVE-2022-21662 Detail

MODIFIED AFTER ENRICHMENT

This CVE record has been updated after NVD enrichment efforts were completed. Enrichment data supplied by the NVD may require amendment due to these changes.

Description

WordPress is a free and open-source content management system written in PHP and paired with a MariaDB database. Low-privileged authenticated users (like author) in WordPress core are able to execute JavaScript/perform stored XSS attack, which can affect high-privileged users. This has been patched in WordPress version 5.8.3. Older affected versions are also fixed via security release, that go back till 3.7.37. We strongly recommend that you keep auto-updates enabled. There are no known workarounds for this issue.

Metrics

CVSS Version 4.0 | CVSS Version 3.x | CVSS Version 2.0

NVD enrichment efforts reference publicly available information to associate vector strings. CVSS information contributed by other sources is also displayed.

CVSS 3.x Severity and Vector Strings:

NIST: NVD	**Base Score:** 5.4 MEDIUM	**Vector:** CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:C/C:L/I:L/A:N
CNA: GitHub, Inc.	**Base Score:** 8.0 HIGH	**Vector:** CVSS:3.1/AV:N/AC:H/PR:L/UI:R/S:C/C:H/I:H/A:H

The NIST Website - Part 2

Read carefully and undestand the description and other information provided:

The NIST Website - Part 2

Read carefully and undestand the description and other information provided:

- Stored **XSS Attack**

The NIST Website - Part 2

Read carefully and undestand the description and other information provided:

- Stored **XSS Attack**
- **Patched** in 5.8.3

The NIST Website - Part 2

Read carefully and undestand the description and other information provided:

- Stored **XSS Attack**
- **Patched** in 5.8.3
- Find affected versions (In this case, from the beginning of Wordpress)
- Find when was introduced

The NIST Website - Part 2

Read carefully and undestand the description and other information provided:

- Stored **XSS Attack**
- **Patched** in 5.8.3
- Find affected versions (In this case, from the beginning of Wordpress)
- Find when was introduced
- Retreive information about the software: programming languages, frameworks, dependancies, etc.

The Data Privacy and Security course

Although you can choose a vulnerability of any kind, selecting one of the types studied during the course will simplify the task.

The Data Privacy and Security course

Although you can choose a vulnerability of any kind, selecting one of the types studied during the course will simplify the task.

> Use the slides provided during the course and review the relevant topics.

Cross Site Scripting (XSS)

What is a XSS Vulnerability?

XSS (**Cross-Site Scripting**) is a vulnerability where an attacker **tricks a website into running their code in another user's browser**.

What is a XSS Vulnerability?

XSS (Cross-Site Scripting) is a vulnerability where an attacker **tricks a website into running their code in another user's browser**.

It works in three steps:

- The attacker **injects HTML or JavaScript** into the website (e.g. via a comment, search box, or URL parameter)
- The website **shows that content to other users**
- The victim's browser **runs it** (because it can't tell attacker code from legitimate site code)

What is a XSS Vulnerability?

XSS (Cross-Site Scripting) is a vulnerability where an attacker **tricks a website into running their code in another user's browser**.

It works in three steps:

- The attacker **injects HTML or JavaScript** into the website (e.g. via a comment, search box, or URL parameter)
- The website **shows that content to other users**
- The victim's browser **runs it** (because it can't tell attacker code from legitimate site code)

Why it matters: the attacker's code runs **as the victim** on that site → it can steal their session cookie, perform actions on their behalf, or modify what they see.

XSS Example: the Attack

Goal: steal the victim's session cookie

1. A user logs into a website → the browser stores a session cookie
2. An attacker **injects** malicious code into the page
3. When the victim visits the page:
   - The code runs in their browser
   - The code **sends their cookie to the attacker's server**
4. The attacker can now impersonate the victim (log in as them by using the stolen cookie)

Start writing your report

You now have all the basic tools to start writing the beginning of your report:

Start writing your report

You now have all the basic tools to start writing the beginning of your report:

- Start by **describing the system** in which you find the vulnerability, in this case WordPress. Dedicate 1–2 pages to this, choosing the appropriate level of abstraction so that the system is understandable and helps explain the vulnerability later.

Start writing your report

You now have all the basic tools to start writing the beginning of your report:

- Start by **describing the system** in which you find the vulnerability, in this case WordPress. Dedicate 1–2 pages to this, choosing the appropriate level of abstraction so that the system is understandable and helps explain the vulnerability later.
- Continue with a chapter on the **vulnerability family**, in this case XSS. You will need to go beyond the slides and conduct thorough research on this type of vulnerability. Dedicate about 2–3 pages to this section. Keep in mind that this is the knowledge you want your reader to have in order to understand your CVE.

Hints on describing the system

During the system description, remember to answer **at least** those questions:

- What is the system you are studing? (Wordpress is an open-source CMS...)
- What can you do with the system? (Wordpress is a blog posting system, a forum, can be used for media galleries, etc.)
- What are the base technological structures? (It uses PHP and MySQL to work)
- How many people are using wordpress? (Over 63 million websites use Wordpress)
- How to use the system in a secure way? (Keep Wordpress updated, do not use untrusted plugins, etc.)

Hints on describing the vulnerability family

During the system description, remember to explain **at least** this points:

- The basic idea about the vulnerability
- Eventual subtypes (Reflected XSS, stored XSS)
- How a generic attack could work
- How to find such vulnerabilities
- How to prevent such vulnerabilities

WordPress: what are we studying?

Before explaining the vulnerability, the report first introduces the affected system.

WordPress: what are we studying?

Before explaining the vulnerability, the report first introduces the affected system.

- WordPress is an open-source **Content Management System** released in 2003
- It is written mainly in **PHP**
- It stores content and configuration in **MySQL** or **MariaDB**
- It supports HTTPS deployments and is typically executed by a web server such as Apache
- Its original goal was blogging, but it is now used for forums, media galleries, online stores, and learning platforms

WordPress as a web application

A useful mental model for the report is: WordPress receives HTTP requests, loads data from the database, applies PHP logic, and returns HTML pages to browsers.

Server side

- PHP core code
- Themes and templates
- Plugins
- Database queries
- Authentication and authorization checks

Client side

- HTML generated by WordPress
- CSS from themes
- JavaScript from core, themes, and plugins
- Browser session cookies
- Admin dashboard pages rendered for privileged users

Themes and plugins

The report emphasizes that extensibility is central to WordPress security.

Themes and plugins

The report emphasizes that extensibility is central to WordPress security.

- **Themes** change the appearance and part of the behavior of a site without editing each page manually
- **Plugins** add features such as SEO tools, private portals, galleries, forms, e-commerce, and security checks
- The plugin/theme ecosystem increases the attack surface
- Outdated components are a common source of WordPress compromise
- Tools such as **WPScan**, WordPress Sploit Framework, and WordPress Auditor are used to search for known weaknesses

WordPress users and privileges

The CVE is interesting because the attacker does not need to be an administrator.

WordPress users and privileges

The CVE is interesting because the attacker does not need to be an administrator.

Relevant roles for this case study

- **Admin**: can manage the site and install plugins
- **Author**: can create and publish posts
- **Subscriber/reader**: has much more limited permissions

CVE-2022-21662 is an authenticated Stored XSS where an **Author** can target an **Admin**.

Posts, URLs, permalinks, and slugs

The vulnerable object is the post slug, so the report explains why slugs exist.

Post

A WordPress content item stored in the database and rendered as a web page.

Permalink

The stable URL used to reach that content.

Slug

The URL-friendly part that identifies the post, usually derived from the title.

Example

```
Title:  My First Security Post
Slug:   my-first-security-post
URL:    https://example.test/my-first-
security-post/
```

The slug becomes security-sensitive because WordPress stores it, normalizes it, and later prints it inside generated pages.

Why slugs are designed this way

Slugs are not just internal identifiers: they are part of the public web interface.

Why slugs are designed this way

Slugs are not just internal identifiers: they are part of the public web interface.

- They help make URLs readable for humans
- They help search engines crawl and index pages
- They often include words from the title, dates, author names, or numeric suffixes
- WordPress lets administrators choose permalink structures
- For the exploit in the report, permalinks must be configured as **Post name**

Why slug handling is security-sensitive

A slug looks like harmless text, but it crosses several trust boundaries.

Why slug handling is security-sensitive

A slug looks like harmless text, but it crosses several trust boundaries.

1. An authenticated user can influence it
2. WordPress sanitizes and normalizes it
3. WordPress stores it in the database
4. WordPress may decode, truncate, or modify it to avoid duplicates
5. WordPress later renders it in HTML admin pages

If one transformation reintroduces dangerous characters, earlier sanitization may become ineffective.

Why slug handling is security-sensitive

A slug looks like harmless text, but it crosses several trust boundaries.

1. An authenticated user can influence it
2. WordPress sanitizes and normalizes it
3. WordPress stores it in the database
4. WordPress may decode, truncate, or modify it to avoid duplicates
5. WordPress later renders it in HTML admin pages

If one transformation reintroduces dangerous characters, earlier sanitization may become ineffective.

> **Report-writing lesson:** when describing an affected program, include just enough architecture to explain where attacker-controlled data enters, where it is stored, and where it is rendered.

The real CVE

Introduction, done!

Let's work on the CVE

CVE-2022-21662: case-study facts

This is the kind of compact technical summary your report should make clear before discussing the exploit.

CVE-2022-21662: case-study facts

This is the kind of compact technical summary your report should make clear before discussing the exploit.

- **Product**: WordPress Core
- **Affected versions**: WordPress up to and including **5.8.2**
- **Fixed version**: WordPress **5.8.3**
- **Vulnerability family**: authenticated **Stored XSS**
- **Vulnerable object**: post **slug** (`post_name`), i.e. the URL-friendly identifier of a post

Where is the bug?

The vulnerable behavior is in the slug generation/normalization path used when a post is inserted or updated.

Normal WordPress flow

- `wp_insert_post()` receives post data
- `$post_name` stores the slug
- `sanitize_title()` transforms the title/slug into a URL-safe value
- `sanitize_title_with_dashes()` preserves URL-encoded octets
- `wp_unique_post_slug()` ensures uniqueness when duplicate slugs exist
- `truncate_post_slug()` shortens the slug before saving it

Security-relevant detail

The dangerous transformation happens after earlier sanitization:

```
URL-encoded slug
  -> urldecode()
  -> utf8_uri_encode(..., 200)
  -> saved as an alternative unique slug
```

In the vulnerable versions, ASCII characters such as <, >, quotes, and / could survive this path.

Why duplicate slugs matter

The exploit abuses the code path that computes an alternative slug when WordPress detects a collision.

Why duplicate slugs matter

The exploit abuses the code path that computes an alternative slug when WordPress detects a collision.

1. The attacker creates two posts, for example `post1` and `post2`
2. The attacker edits the slug of `post1` with a URL-encoded JavaScript payload plus padding
3. The attacker sets the slug of `post2` to the same value
4. WordPress detects the duplicate and calls the uniqueness/truncation logic
5. The URL-encoded payload is decoded and stored in a form that reaches an HTML response

Why duplicate slugs matter

The exploit abuses the code path that computes an alternative slug when WordPress detects a collision.

1. The attacker creates two posts, for example `post1` and `post2`
2. The attacker edits the slug of `post1` with a URL-encoded JavaScript payload plus padding
3. The attacker sets the slug of `post2` to the same value
4. WordPress detects the duplicate and calls the uniqueness/truncation logic
5. The URL-encoded payload is decoded and stored in a form that reaches an HTML response

Key point: the payload is not only reflected once. It is stored in WordPress data and later rendered when another user opens the affected admin page.

Payload shape

The report uses an alert payload to demonstrate execution without causing real damage.

Payload shape

The report uses an alert payload to demonstrate execution without causing real damage.

A minimal conceptual payload is:

```
"/><script>alert("Vulnerability Found")</script>
```

For the real trigger, the payload is **URL-encoded** and padded so that the slug-processing path involving the 200-character limit and duplicate-slug handling is reached.

Exploit: why the payload survives

The bug is not simply “WordPress forgot sanitization”. It is a **decode-after-sanitize** problem.

Exploit: why the payload survives

The bug is not simply “WordPress forgot sanitization”. It is a **decode-after-sanitize** problem.

Step 1: sanitization preserves URL octets

`sanitize_title_with_dashes()` removes many dangerous characters, but it intentionally preserves URL-encoded octets such as `%3C` and `%3E` because encoded bytes can be valid in URLs.

```
%22%2F%3E%3Cscript%3Ealert(1)%3C%2Fscript%3E
```

can therefore pass as slug-like data.

Exploit: why the payload survives

The bug is not simply “WordPress forgot sanitization”. It is a **decode-after-sanitize** problem.

Step 1: sanitization preserves URL octets

`sanitize_title_with_dashes()` removes many dangerous characters, but it intentionally preserves URL-encoded octets such as `%3C` and `%3E` because encoded bytes can be valid in URLs.

```
%22%2F%3E%3Cscript%3Ealert(1)%3C%2Fscript%3E
```

can therefore pass as slug-like data.

Step 2: duplicate slug handling decodes it

When the second post reuses the same long encoded slug, WordPress computes a unique alternative slug and calls `_truncate_post_slug()`.

```
encoded slug -> urldecode() -> decoded HTML/JS payload
```

The vulnerable version then re-encodes only Unicode characters, not ASCII metacharacters.

Original vulnerable code

In WordPress 5.8.2, `_truncate_post_slug()` decodes the slug and then calls `utf8_uri_encode()` without requesting ASCII encoding.

Original vulnerable code

In WordPress 5.8.2, `_truncate_post_slug()` decodes the slug and then calls `utf8_uri_encode()` without requesting ASCII encoding.

```
function _truncate_post_slug( $slug, $length = 200 ) {
    if ( strlen( $slug ) > $length ) {
        $decoded_slug = urldecode( $slug );
        if ( $decoded_slug === $slug ) {
            $slug = substr( $slug, 0, $length );
        } else {
            $slug = utf8_uri_encode( $decoded_slug, $length );
        }
    }

    return rtrim( $slug, '-' );
}
```

Why this is exploitable

`utf8_uri_encode()` was originally designed to encode Unicode bytes for URLs, not to make arbitrary decoded ASCII safe for HTML.

Why this is exploitable

`utf8_uri_encode()` was originally designed to encode Unicode bytes for URLs, not to make arbitrary decoded ASCII safe for HTML.

Important consequence

`php utf8uriencode('', 200 )`

returns a string where ASCII characters like <, >, /, quotes, and letters are not encoded.

So a slug that started as URL-encoded text can become stored HTML/JavaScript.

Exploit flow in the database

The exploit needs the database state to contain the decoded payload as a post slug.

First post

- Set slug to the long URL-encoded payload
- No collision yet
- It remains stored as a slug value

Second post

- Set slug to the same value
- Collision triggers `wp_unique_post_slug()`
- Alternative slug calculation triggers `_truncate_post_slug()`
- The payload is decoded and stored

Trigger

- Admin opens the post listing page
- WordPress prints the slug assuming it is safe
- Browser interprets injected markup/script
- Script executes in the admin session

This is why the vulnerability is **stored** and why the victim is not the attacker.

From alert to real impact

The report uses `alert()` only as a safe proof of execution.

From alert to real impact

The report uses `alert()` only as a safe proof of execution.

A real payload would target actions available to the victim, for example:

- send authenticated requests from the admin browser
- create or modify content
- change configuration
- upload or activate a malicious plugin if the victim has permissions
- reach arbitrary PHP execution through plugin functionality

The report should clearly separate **proof of concept** from **realistic impact**.

Attack prerequisites and victim

Not every WordPress user can exploit this CVE.

Attacker

- Must be authenticated
- Needs permissions to create/edit posts
- In the report scenario: an **Author** account

Victim

- A higher-privileged user visiting the posts administration area
- In the report scenario: an **Admin** account

Impact chain

1. Stored JavaScript executes in the victim browser
2. The script runs with the victim session
3. The attacker can perform actions available to the victim
4. With an admin victim, this may lead to malicious plugin upload and arbitrary server-side PHP execution

Reproducibility checklist for the report

A strong project report does not just say that the CVE exists: it shows how to reproduce it.

Reproducibility checklist for the report

A strong project report does not just say that the CVE exists: it shows how to reproduce it.

- Start WordPress **5.8.2** with Docker Compose
- Configure permalinks as **Post name**
- Create at least two users: one **Admin**, one **Author**
- As Author, create two posts
- Modify both slugs through **Quick Edit**
- Use the crafted URL-encoded and padded payload
- As Admin, visit the posts dashboard and observe script execution
- Repeat on WordPress **5.8.3** and show that the payload is encoded instead of executed

Patch idea in WordPress 5.8.3

The fix is a good example of context-sensitive output handling.

Patch idea in WordPress 5.8.3

The fix is a good example of context-sensitive output handling.

- WordPress changed the slug encoding path used by `utf8_uri_encode()`
- A new boolean option allows encoding ASCII characters too
- When enabled, non-alphanumeric characters are encoded with `rawurlencode()`
- Characters needed to break out of HTML/attribute context no longer survive as executable markup

Patch idea in WordPress 5.8.3

The fix is a good example of context-sensitive output handling.

- WordPress changed the slug encoding path used by `utf8_uri_encode()`
- A new boolean option allows encoding ASCII characters too
- When enabled, non-alphanumeric characters are encoded with `rawurlencode()`
- Characters needed to break out of HTML/attribute context no longer survive as executable markup

Lesson: sanitizing a value once is not enough if later transformations decode, truncate, concatenate, or otherwise change that value before it is rendered.

Patched code: `_truncate_post_slug()`

The WordPress patch is intentionally small at the call site.

Patched code: `_truncate_post_slug()`

The WordPress patch is intentionally small at the call site.

```
- $slug = utf8_uri_encode( $decoded_slug, $length );
+ $slug = utf8_uri_encode( $decoded_slug, $length, true );
```

The third argument tells `utf8_uri_encode()` to encode ASCII metacharacters too.

Patched code: `utf8_uri_encode()`

WordPress 5.8.3 adds the `$encode_ascii_characters` parameter.

Patched code: `utf8_uri_encode()`

WordPress 5.8.3 adds the `$encode_ascii_characters` parameter.

```
-function utf8_uri_encode( $utf8_string, $length = 0 ) {
+function utf8_uri_encode( $utf8_string, $length = 0,
$encode_ascii_characters = false ) {
```

When the parameter is `false`, existing callers keep the old behavior. When it is `true`, ASCII characters are encoded as part of the output length accounting.

Patch internals

For ASCII bytes, the patched function conditionally applies `rawurlencode()`.

Patch internals

For ASCII bytes, the patched function conditionally applies `rawurlencode()`.

```
if ( $value < 128 ) {
    $char                = chr( $value );
    $encoded_char        = $encode_ascii_characters ? rawurlencode( $char
) : $char;
    $encoded_char_length = strlen( $encoded_char );

    if ( $length && ( $unicode_length + $encoded_char_length ) > $length )
{
        break;
    }

    $unicode        .= $encoded_char;
    $unicode_length += $encoded_char_length;
}
```

Mitigation: what changes after 5.8.3?

The dangerous decoded payload is converted back to URL-safe bytes before storage/rendering.

Before patch

```
%3Cscript%3Ealert(1)%3C/script%3E
  -> urldecode()
<script>alert(1)</script>
  -> utf8_uri_encode(..., 200)
<script>alert(1)</script>
```

After patch

```
<script>alert(1)</script>
  -> utf8_uri_encode(..., 200, true)
%3Cscript%3Ealert%281%29%3C%2Fscript%3E
```

Security lesson

The patch does not rely on the admin page escaping the value later. It fixes the invariant expected from slugs: after truncation and uniqueness handling, the slug must still be URL-safe.

This is defense at the data-normalization boundary.

Mitigation discussion for the report

Students should discuss both the immediate fix and operational mitigations.

Mitigation discussion for the report

Students should discuss both the immediate fix and operational mitigations.

- **Code fix**: upgrade WordPress Core to **5.8.3** or later
- **Validation**: repeat the same exploit steps on 5.8.3 and show that JavaScript does not execute
- **Operational control**: do not grant Author privileges to untrusted users
- **Hardening**: keep core, themes, and plugins updated
- **Defense in depth**: admin pages should still escape untrusted output even when data is expected to be sanitized

Disclosure timeline

Your report should explain not only the bug, but also when it was reported and fixed.

Disclosure timeline

Your report should explain not only the bug, but also when it was reported and fixed.

Date	Event
18-10-2018	Issue reported to WordPress through HackerOne
26-11-2018	WordPress confirmed the problem
29-10-2020	WordPress 5.5.2 changelog suggested a fix
28-12-2020	Community reported that the issue was still exploitable
24-02-2021	WordPress planned a fix for a future release
03-12-2021	Disclosure deadline communicated
06-01-2022	WordPress 5.8.3 fixed the vulnerability

What makes this a good project example?

CVE-2022-21662 maps directly to the required sections of the project report.

What makes this a good project example?

CVE-2022-21662 maps directly to the required sections of the project report.

- **Affected program**: WordPress architecture, users, posts, slugs, database storage
- **Vulnerability family**: Stored XSS and server/client-side XSS classification
- **Specific CVE**: vulnerable slug-processing path and duplicate-slug trigger
- **Exploitation**: author-to-admin stored payload execution
- **Reproduction**: Docker Compose with WordPress 5.8.2 and MySQL
- **Mitigation**: WordPress 5.8.3 encoding changes
- **Fix reproduction**: same attack against patched version should not execute JavaScript

What should you know about your CVE?

What should you know about your CVE?

EVERYTHING THERE IS TO KNOW!

What should you know about your CVE?

EVERYTHING THERE IS TO KNOW!

Just joking! :)

What should you know about your CVE?

EVERYTHING THERE IS TO KNOW!

Just joking! :)

MORE than everything

What should you know about your CVE?

EVERYTHING THERE IS TO KNOW!

Just joking! :)

MORE than everything

You'll need to become the world finest expert on that CVE

Explain your vulnerability

This is the most important portion of your report.

Explain your vulnerability

This is the most important portion of your report.

You will need to find your sources... or get help by "a friend"

Explain your vulnerability

This is the most important portion of your report.

You will need to find your sources... or get help by "a friend"

> But remember: you will need to UNDERSTAND what you've written in your report.
>
> **Questions about your report will be in the written part.**

Hints on describing the CVE

During the CVE specific chapter, remember to explain these points, some of them could not apply to your vulnerability, you will need to adapt them specifically:

- Who can exploit the CVE (authenticated users, unauthenticated users, users with access to specific portions, etc.)
- The CVE impact (all Wordpress instances? instances with a specific plugin installed? instances on a specific operating system? does it need a specific setting to be enabled?)
- The "danger score" of the CVE (what additional privileges will the attacker have?)
- The specific portion of the source code regarding the vulnerability
- Possible attack strategies
- The remedy (what was needed to fix this CVE? can you point out the specific lines of code that fix the vulnerability?)
- How much time passed between the CVE and the patch?

Hints on describing the CVE

During the CVE specific chapter, remember to explain these points, some of them could not apply to your vulnerability, you will need to adapt them specifically:

- Who can exploit the CVE (authenticated users, unauthenticated users, users with access to specific portions, etc.)
- The CVE impact (all Wordpress instances? instances with a specific plugin installed? instances on a specific operating system? does it need a specific setting to be enabled?)
- The "danger score" of the CVE (what additional privileges will the attacker have?)
- The specific portion of the source code regarding the vulnerability
- Possible attack strategies
- The remedy (what was needed to fix this CVE? can you point out the specific lines of code that fix the vulnerability?)
- How much time passed between the CVE and the patch?

> These points are **only guidelines**. Some of them may not apply to your case, and for certain CVEs there may be no answer to some of them. Some CVEs may require additional or different points. **Do not strictly follow these points**; use them as a starting point instead.

Reproducibility

Baseline

To achieve the reproducibility goal, you will need to complete all of the following points:

Baseline

To achieve the reproducibility goal, you will need to complete all of the following points:

1. Create a setup with the software in it's vulnerable version on your machine

Baseline

To achieve the reproducibility goal, you will need to complete all of the following points:

1. Create a setup with the software in it's vulnerable version on your machine
2. Exploit the CVE by successfully attacking the software

Baseline

To achieve the reproducibility goal, you will need to complete all of the following points:

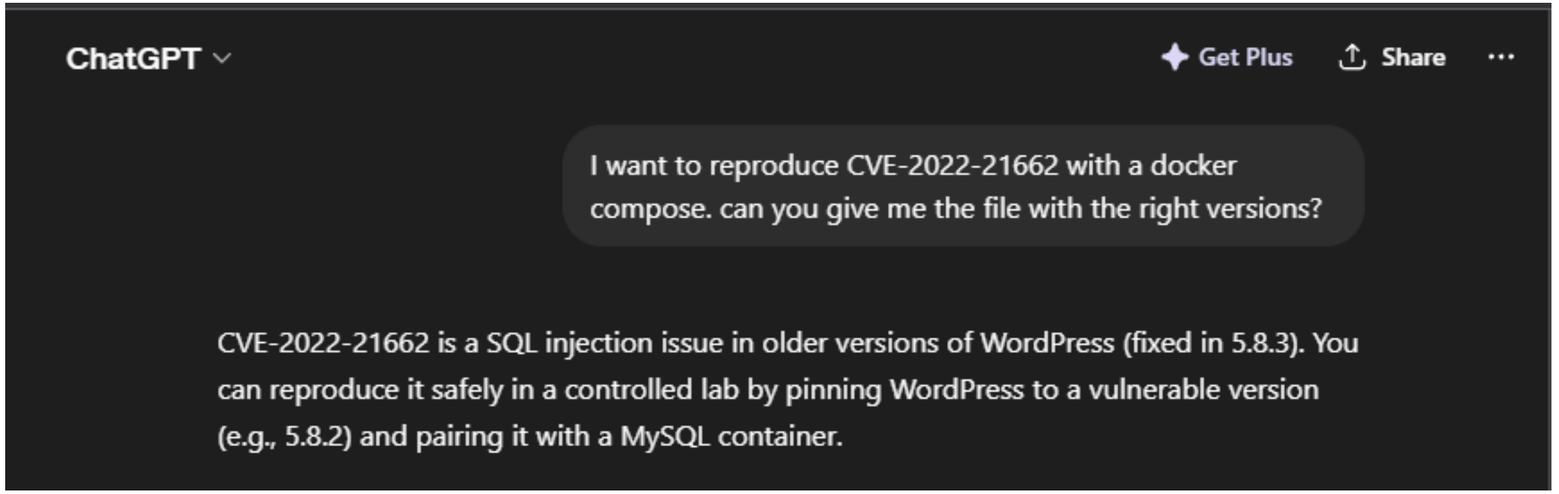
1. Create a setup with the software in it's vulnerable version on your machine
2. Exploit the CVE by successfully attacking the software
3. Share the relevant files with your professors (that's the easy part)

Vulnerable setup

The best way to create a reproducible vulnerable setup on your machine is by using Docker. Since many tools have requirements (e.g., WordPress needs a database system), a Docker Compose setup is even better.

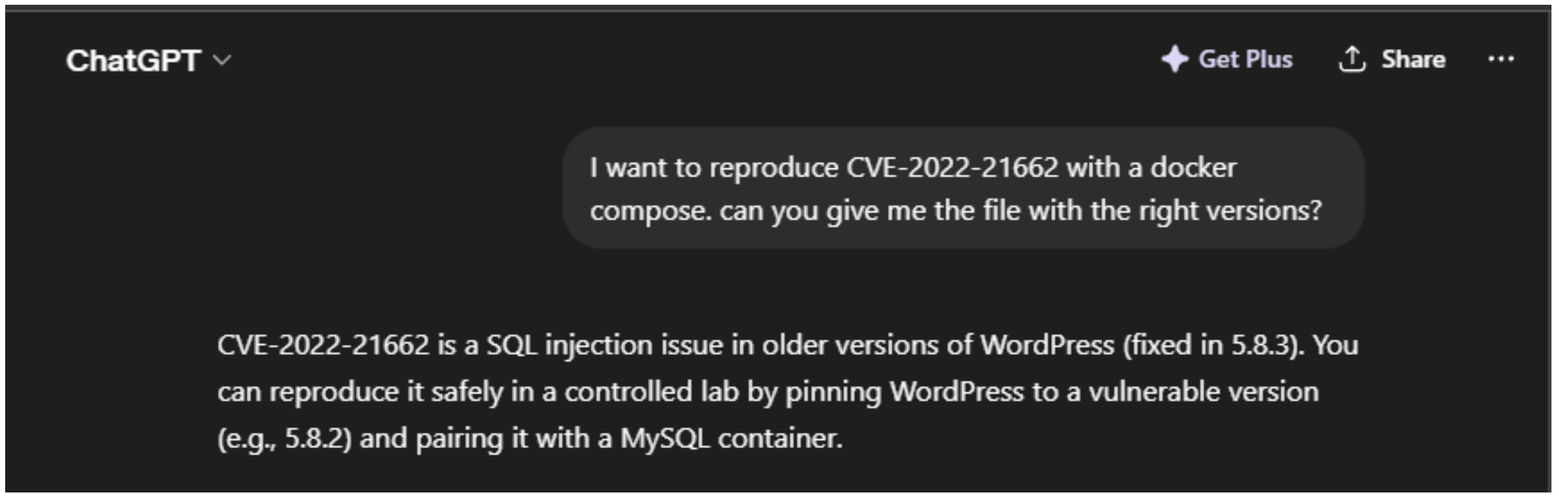
Vulnerable setup

The best way to create a reproducible vulnerable setup on your machine is by using Docker. Since many tools have requirements (e.g., WordPress needs a database system), a Docker Compose setup is even better.



Vulnerable setup

The best way to create a reproducible vulnerable setup on your machine is by using Docker. Since many tools have requirements (e.g., WordPress needs a database system), a Docker Compose setup is even better.



Remember: you will need to UNDERSTAND what's inside your Docker Compose file

Use the vulnerable setup

Start by using the software in a legitimate way.

Understand how it works, it's basic features, get to know your privileges.

Use the vulnerable setup

Start by using the software in a legitimate way.

Understand how it works, it's basic features, get to know your privileges.

It will be easier to exploit your vulnerability if you know how the software works (from the user point of view)

Docker Compose Example

```yaml
services:
  db:
    image: mysql:5.7
    environment:
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wp_user
      MYSQL_PASSWORD: wp_pass
      MYSQL_ROOT_PASSWORD: root_pass
    volumes:
      - db_data:/var/lib/mysql

  wordpress:
    image: wordpress:5.8.2-php7.4-apache
    depends_on:
      - db
    ports:
      - "8080:80"
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wp_user
      WORDPRESS_DB_PASSWORD: wp_pass
      WORDPRESS_DB_NAME: wordpress
    volumes:
      - wp_data:/var/www/html
```

Docker reproduction: what to submit

The reproduction environment should let the evaluator run both the vulnerable case and the patched case.

Docker reproduction: what to submit

The reproduction environment should let the evaluator run both the vulnerable case and the patched case.

A good submission includes:

- `docker-compose.yml` for WordPress **5.8.2**
- a second compose file or documented edit for WordPress **5.8.3**
- database credentials and exposed port
- exact browser steps to initialize WordPress
- exact users/roles to create
- exact permalink setting
- exact payload-generation command or script
- screenshots or logs proving vulnerable and patched behavior

Docker reproduction: initialization

After `docker compose up`, the site still needs deterministic manual setup.

Docker reproduction: initialization

After `docker compose up`, the site still needs deterministic manual setup.

1. Open `http://localhost:8080`
2. Complete the WordPress installation wizard
3. Create the administrator account
4. Go to **Settings** → **Permalinks**
5. Select **Post name**
6. Create an Author user, for example `author`
7. Log out from Admin and log in as Author

Document these steps because they are part of the exploit preconditions.

Docker reproduction: vulnerable run

The evaluator should be able to reproduce the XSS without guessing the missing steps.

Docker reproduction: vulnerable run

The evaluator should be able to reproduce the XSS without guessing the missing steps.

1. As Author, create two posts: `post1` and `post2`
2. Generate the URL-encoded payload with enough padding to exceed 200 characters
3. Open **Posts** → **All Posts**
4. Use **Quick Edit** on `post1` and set its slug to the generated value
5. Use **Quick Edit** on `post2` and set the same slug
6. Log out from Author
7. Log in as Admin
8. Open **Posts** → **All Posts**
9. Observe the proof-of-execution alert

Example payload generator

The report should include the exact payload-generation logic, not only a screenshot.

Example payload generator

The report should include the exact payload-generation logic, not only a screenshot.

```
from urllib.parse import quote

payload = "\"/><script>alert(\"Vulnerability Found\")</script>"
encoded = quote(payload, safe="")
slug = encoded + ("a" * max(0, 210 - len(encoded)))

print(slug)
print("length:", len(slug))
```

The padding is not the attack itself: it is used to reach the `_truncate_post_slug()` path where the vulnerable decode/re-encode behavior occurs.

Docker reproduction: patched run

The patched reproduction is as important as the vulnerable one.

Docker reproduction: patched run

The patched reproduction is as important as the vulnerable one.

Repeat the same workflow with:

```
wordpress:
  image: wordpress:5.8.3-php7.4-apache
```

Expected result:

- the same payload does **not** execute
- the stored slug remains URL-encoded/safe
- the Admin post listing renders text/URL data instead of script markup

This demonstrates that the mitigation addresses the root cause.

Docker reporting checklist

For grading, make the reproduction boring and deterministic.

Docker reporting checklist

For grading, make the reproduction boring and deterministic.

- State Docker and Docker Compose versions used
- Pin image tags and ports
- Explain how to reset state: remove the named volumes and restart
- Include credentials only for the local test environment
- Include the vulnerable evidence and the patched evidence
- Explain every manual WordPress configuration step
- Explain why each precondition matters: Author role, duplicate slugs, `Post name` permalinks, and payload length

Exploit your vulnerability

In order to exploit your vulnerability you have two options:

- You understand perfectly how the vulnerability works, you understood how to exploit such vulnerability during the lessons, you are familiar with the programming languages involved, you can write a script to exploit the vulnerability
- You ask "a friend" to help you exploit it

Exploit your vulnerability

In order to exploit your vulnerability you have two options:

- You understand perfectly how the vulnerability works, you understood how to exploit such vulnerability during the lessons, you are familiar with the programming languages involved, you can write a script to exploit the vulnerability
- You ask "a friend" to help you exploit it

> Most LLMs will not automatically help you to exploit the vulnerability but I'm sure you'll find a way

Exploit your vulnerability

In order to exploit your vulnerability you have two options:

- You understand perfectly how the vulnerability works, you understood how to exploit such vulnerability during the lessons, you are familiar with the programming languages involved, you can write a script to exploit the vulnerability
- You ask "a friend" to help you exploit it

Most LLMs will not automatically help you to exploit the vulnerability but I'm sure you'll find a way

Remember to document all your steps needed to perpetrate the attack

Exploit your vulnerability - Part 2

In addition to the attack, you will need to explain all the required steps for your attack to work:

- Do you need to create different users? With different privileges?
- Do you need specific backend settings to be enable in order to be vulnerable?
- Do you need to do **anything** else from the `docker compose up` to the attack itself? Why?

Exploit your vulnerability - Part 2

In addition to the attack, you will need to explain all the required steps for your attack to work:

- Do you need to create different users? With different privileges?
- Do you need specific backend settings to be enable in order to be vulnerable?
- Do you need to do **anything** else from the `docker compose up` to the attack itself? Why?

> Generically speaking, you will need to explain all the steps necessary to be done before the attack

Exploit your vulnerability - Part 3

About the attack itself, it could be a script, it could be a series of steps to be taken manually, the important part is that it works!

Remember to explain everything in your report and to understand what you are submitting.

Exploit your vulnerability - Part 3

About the attack itself, it could be a script, it could be a series of steps to be taken manually, the important part is that it works!

Remember to explain everything in your report and to understand what you are submitting.

> And that's why choosing the right CVE is the most important step!


That's all Folks!